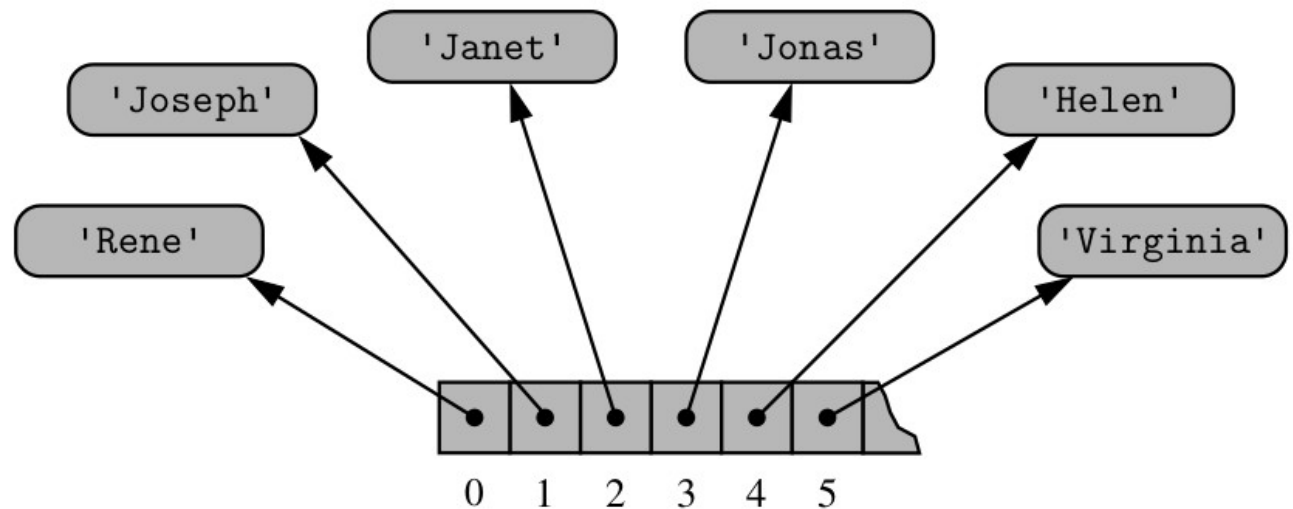
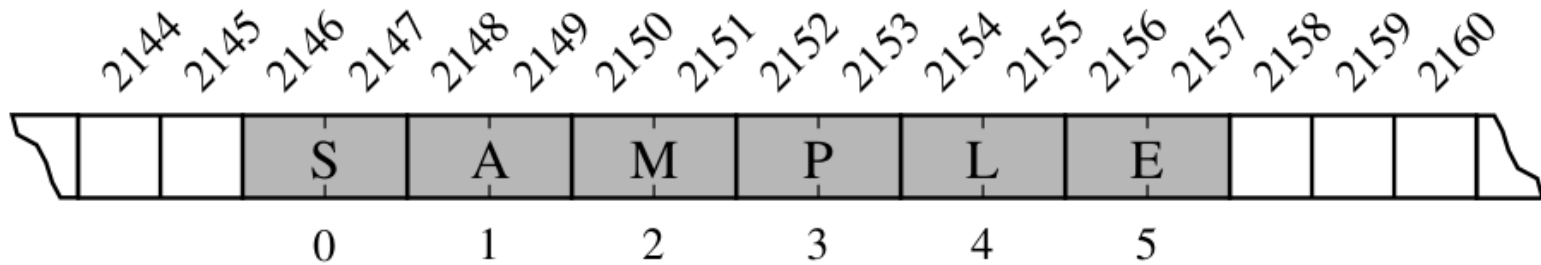
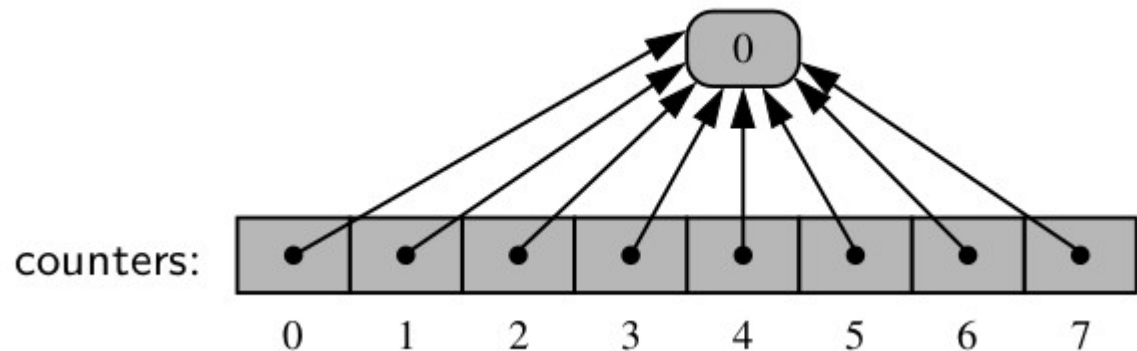
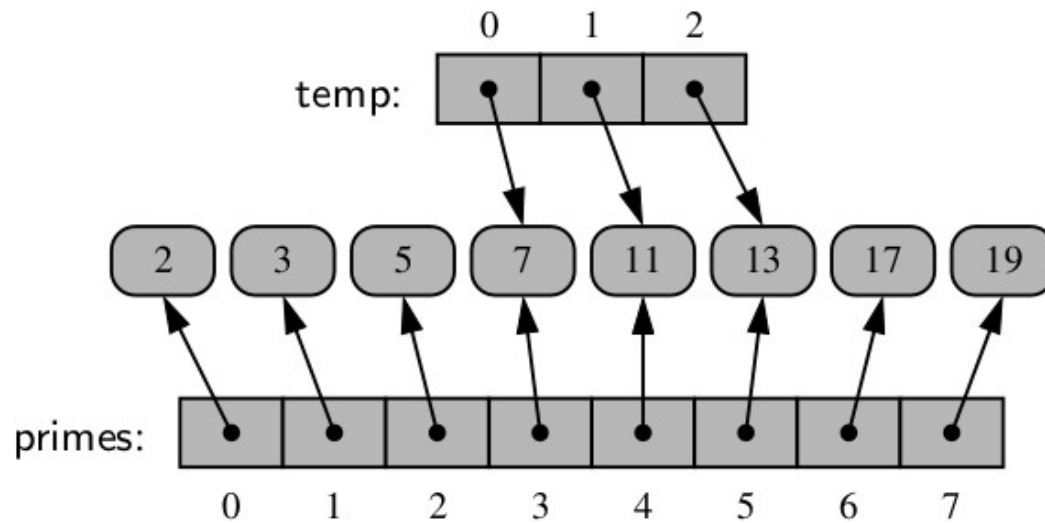


Array based Sequences

Compact and referential arrays



Referential arrays



Type Codes in the array Class

Python's array class has the following type codes:

Code	C Data Type	Typical Number of Bytes
'b'	signed char	1
'B'	unsigned char	1
'u'	Unicode char	2 or 4
'h'	signed short int	2
'H'	unsigned short int	2
'i'	signed int	2 or 4
'I'	unsigned int	2 or 4
'l'	signed long int	4
'L'	unsigned long int	4
'f'	float	4
'd'	float	8

Compact Arrays

```
import array as arr
```

```
# Array of floats:
```

```
farray = arr.array('f', [1.21, 3.054, 7.9])
```

```
# Array of characters:
```

```
c_array = arr.array('u', 'hello') # 'u' for Unicode characters
```

```
# Array of integers:
```

```
i_array = arr.array('i', [100, 2, 230, 40]) # 'i' for signed  
integers
```

Numpy library can also be used for using
compact arrays

Dynamic array implementation

To add an element to an array (without an index), we could do it at the end of the array

When the array is full, we can replace the array with a larger one

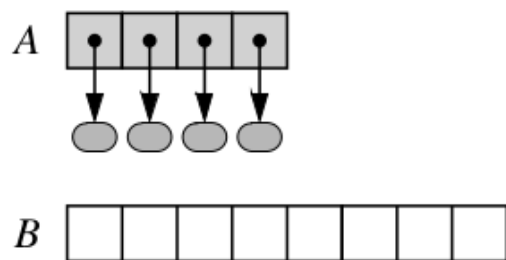
But how large should the new array be?

Incremental strategy: increase the size by a constant c

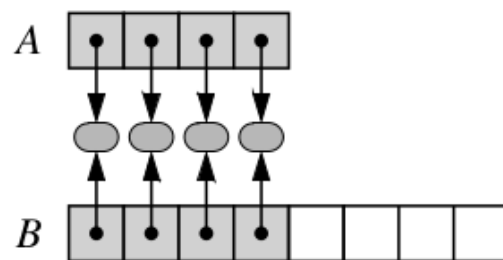
Doubling strategy: double the size

Steps to implement dynamic arrays

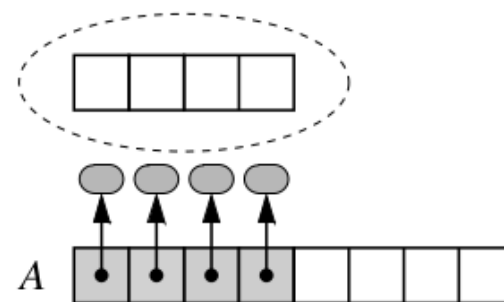
1. Allocate a new array B with larger capacity.
2. Set $B[i] = A[i]$, for $i = 0, \dots, n - 1$, where n denotes current number of items.
3. Set $A = B$, that is, we henceforth use B as the array supporting the list.
4. Insert the new element in the new array.



(a)



(b)



(c)

Incremental Strategy Analysis

To be proved in the class

Time complexity for n insertions is $\Omega(n^2)$

The amortized time of an add operation is $\Omega(n)$

Doubling Strategy Analysis

To be proved in the class

The total time $T(n)$ of a series of n add operations is proportional to

$$\sim 3n$$

$T(n)$ is $O(n)$

The amortized time of an add operation is $O(1)$

```
import sys
data = [ ]
for k in range(100):
    length = len(data) # number of elements

    b = sys.getsizeof(data) # actual size in bytes

    print( f" array length is {length} Size in bytes: {b}" )
    data.append(None)
```

Operation	Running Time
<code>len(data)</code>	$O(1)$
<code>data[j]</code>	$O(1)$
<code>data.count(value)</code>	$O(n)$
<code>data.index(value)</code>	$O(k + 1)$
<code>value in data</code>	$O(k + 1)$
<code>data1 == data2</code> (similarly <code>!=</code> , <code><</code> , <code><=</code> , <code>></code> , <code>>=</code>)	$O(k + 1)$
<code>data[j:k]</code>	$O(k - j + 1)$
<code>data1 + data2</code>	$O(n_1 + n_2)$
<code>c * data</code>	$O(cn)$

Asymptotic performance of the nonmutating behaviors of the list and tuple classes.

Identifiers `data`, `data1`, and `data2` designate instances of the list or tuple class, and n , n_1 , and n_2 their respective lengths.

For the index method and containment check, k represents the index of the leftmost occurrence (with $k = n$ if there is no occurrence). For comparisons between two sequences, we let k denote the leftmost index at which they disagree or else $k = \min(n_1, n_2)$.

Average running time of append

n	100	1,000	10,000	100,000	1,000,000	10,000,000	100,000,000
μs	0.219	0.158	0.164	0.151	0.147	0.147	0.149

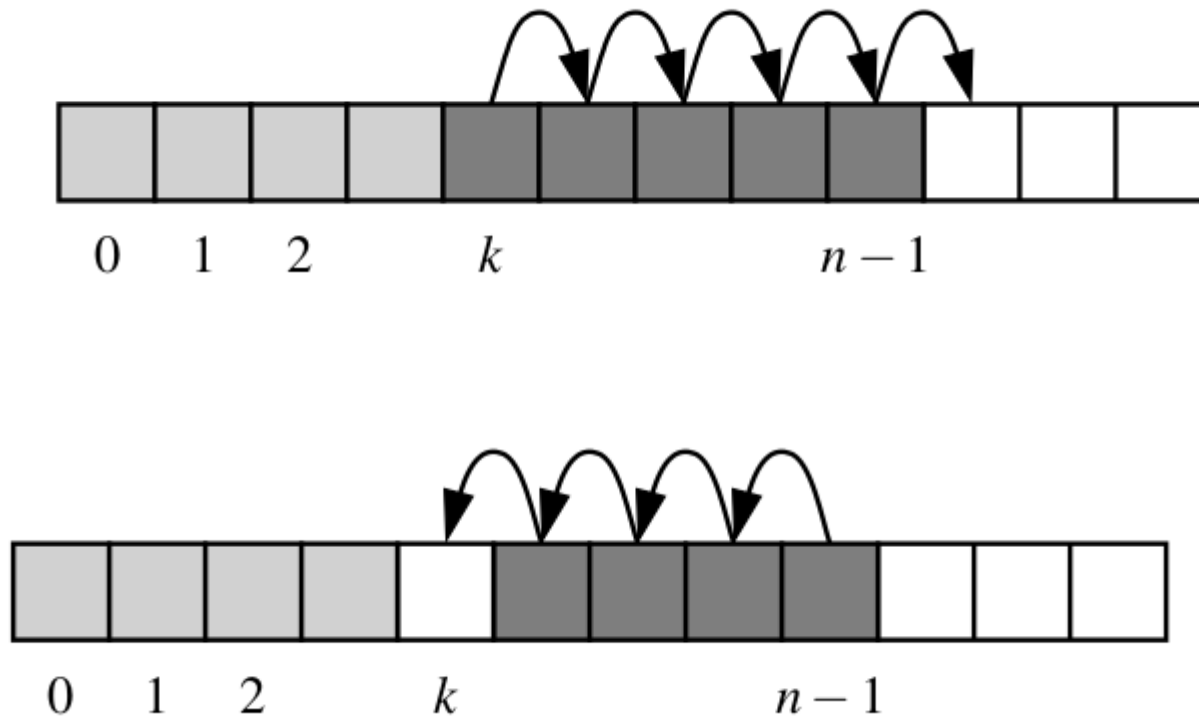
Average running time of append, measured in microseconds, as observed over a sequence of n calls, starting with an empty list.

Operation	Running Time
<code>data[j] = val</code>	$O(1)$
<code>data.append(value)</code>	$O(1)^*$
<code>data.insert(k, value)</code>	$O(n - k + 1)^*$
<code>data.pop()</code>	$O(1)^*$
<code>data.pop(k)</code> <code>del data[k]</code>	$O(n - k)^*$
<code>data.remove(value)</code>	$O(n)^*$
<code>data1.extend(data2)</code> <code>data1 += data2</code>	$O(n_2)^*$
<code>data.reverse()</code>	$O(n)$
<code>data.sort()</code>	$O(n \log n)$

*amortized

Asymptotic performance of the mutating behaviors of the list class. Identifiers `data`, `data1`, and `data2` designate instances of the list class, and n , n_1 , and n_2 their respective lengths.

Insertion & Removal



In the worst case ($i = 0$), both take $O(n)$ time

Average running time of insert(k , val) (measured in microseconds)

	N				
	100	1,000	10,000	100,000	1,000,000
$k = 0$	0.482	0.765	4.014	36.643	351.590
$k = n // 2$	0.451	0.577	2.191	17.873	175.383
$k = n$	0.420	0.422	0.395	0.389	0.397

Average running time of insert(k , val), measured in microseconds, as observed over a sequence of N calls, starting with an empty list. The variable n denotes the size of the current list (as opposed to the final list).

Performance

In an array based implementation of a dynamic list:

The space used by the data structure is $O(n)$

Indexing the element at I takes $O(1)$ time

add and *remove* run in $O(n)$ time in worst case

In an *add* operation, when the array is full, instead of throwing an exception, we can replace the array with a larger one...